

Docker with PHP and Nginx

Use Compose to run two services together.

Learning Goals

- Explain why one application can need several containers (Nginx and PHP-FPM).
- Read `docker-compose.yml` and start the services with `docker compose up -d --build`.
- Follow one request from Nginx to PHP-FPM, and check, log, and stop the services.

The Problem

In the project, we need to run multiple containers:

- PHP-FPM to process PHP requests
- NGINX to serve web requests and forward PHP requests
- MySQL database server

The Solution - Compose

Docker Compose orchestrates multiple containers as one application.

The Example

Copy `module3/code/1_Docker/3. Docker with PHP and NGINX/`
`simple-php-nginx/` to a practice directory.

The Compose File: docker-compose.yml

```
services:
  # NGINX Web Server
  nginx:
    image: nginx:alpine
    container_name: web-server
    ports:
      - "8000:80"
    volumes:
      - ./www:/var/www/html
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf
    depends_on:
      - php-app

  # PHP Application
  php-app:
    build:
      context: .
      dockerfile: php/Dockerfile
    container_name: php-container
    volumes:
      - ./www:/var/www/html
```

In this YAML file, we orchestrate two docker containers.

- `nginx` is the web server that receives HTTP requests.
- `php-app` runs PHP-FPM to process PHP requests.

We will explain how each container works in the next section.

Build and Run

```
docker compose up -d --build
```

- `-d` runs the containers in the background and returns control to the terminal.
- `--build` builds the service images before starting the containers.

✓ Image nginx:alpine	Pulled	5.5s
✓ Image simple-php-nginx-php-app	Built	30.6s
✓ Network simple-php-nginx_default	Created	0.0s
✓ Container php-container	Started	0.2s
✓ Container web-server	Started	

Check the containers

```
docker compose ps
```

This command lists the containers in the Compose project and their status:

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
php-container	simple-php-nginx-php-app	"docker-php-entrypoi..."	php-app	2 minutes ago	Up 2 minutes	9000/tcp
web-server	nginx:alpine	"/docker-entrypoint..."	nginx	2 minutes ago	Up 2 minutes	0.0.0.0:8000->80/tcp, [::]:8000->80/tcp

Test

```
curl -i http://localhost:8000/test.php
```

```
HTTP/1.1 200 OK
Server: nginx/1.31.6
Date: Thu, 01 Oct 2026 20:09:45 GMT
Content-Type: application/json
Transfer-Encoding: chunked
Connection: keep-alive
X-Powered-By: PHP/8.4.26

{
  "message": "Hello from PHP Container!",
  "php_version": "8.4.26",
  "timestamp": "2026-10-01T20:09:45+00:00",
  "server": "nginx\1.31.6",
  "container": "68f5951606b4"
}%
```

Follow One Request

```
localhost:8000 → nginx:80 → php-app:9000 → www/test.php
```

- Nginx serves static files and forwards PHP requests.
- PHP-FPM executes PHP.
- Both containers mount `www/` at `/var/www/html`.

Files and Responsibilities

File	Purpose
<code>docker-compose.yml</code>	Services, ports, and shared directories
<code>php/Dockerfile</code>	PHP-FPM runtime
<code>nginx/default.conf</code>	Route requests to PHP
<code>www/</code>	PHP and HTML source

Compose builds from the project root, so the Dockerfile uses

```
COPY www .
```

Nginx

The NGINX container loads `nginx/default.conf` to route PHP requests to PHP-FPM.

docker-compose.yml: NGINX service

The `nginx` service defines the NGINX container and its settings.

- It downloads the nginx image from docker repository.
- The name of the container is `web-server` and it depends on `php-app` container.
- The 8000 port is matched to the internal 80 port.

```
nginx:
  image: nginx:alpine
  container_name: web-server
  ports:
    - "8000:80"
  volumes:
    - ./www:/var/www/html
    - ./nginx/default.conf:/etc/nginx/conf.d/default.conf
  depends_on:
    - php-app
```

- `volumes` bind-mounts the local `www/` directory and NGINX configuration file into the container.
- For example, `./www:/var/www/html` matches current `.www` directory matches container's `/var/www/html` directory.

How Nginx Finds PHP

We use the same NGINX configuration that we used in the module 1.

```
root /var/www/html;
location ~ /\.php$ {
    include fastcgi_params;
    fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
    fastcgi_pass php-app:9000;
}
```

`php-app` is the Compose service name. Use the complete supplied config.

PHP

The PHP container uses the Docker file that we discussed in the previous section.

```
# PHP Application
php-app:
  build:
    context: .
    dockerfile: php/Dockerfile
  container_name: php-container
  volumes:
    - ./www:/var/www/html
```

PHP Dockerfile

We use the same PHP8.4.

```
# Use official PHP with FPM (FastCGI Process Manager)  
FROM php:8.4-fpm
```

We set the working directory in the container as the WORKDIR.

```
# Set working directory  
WORKDIR /var/www/html
```

We install PHP extensions for MySQL.

```
# Install basic PHP extensions  
RUN docker-php-ext-install pdo pdo_mysql
```

`context: .` means the directory containing this `docker-compose.yml` (`simple-php-nginx/`).

- Docker uses that directory as the build context, so `COPY` paths start there, not in `php/` .

```
php-app:
  build:
    context: .
```

The `./www` volume path is also relative to the directory containing `docker-compose.yml`; it bind-mounts that host folder at `/var/www/html` while the container is running.

```
volumes:  
  - ./www:/var/www/html
```

```
# Set proper permissions  
RUN chown -R www-data:www-data /var/www/html
```

The NGINX container communicates using the php-app:9000, so we expose the port.

```
location ~ /\.php$ {  
    fastcgi_pass php-app:9000; # Container name  
    fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;  
    include fastcgi_params;  
}
```

```
# Expose PHP-FPM port  
EXPOSE 9000
```

Finally, we start the php-fpm process in the containers.

```
# Start PHP-FPM  
CMD ["php-fpm"]
```

The www directory

The .www local directory has three PHP scripts.

- In the container, they are mapped to `/var/www/html` that is the `WORKDIR` of the PHP with FPM.
- Because of the `context: .` setup, `COPY ./www` specifies the www directory in the `simple-php-nginx`.

Make a Change

```
docker compose restart nginx  
docker compose up -d --build
```

- `docker compose restart nginx` restarts only the NGINX container so it loads the updated configuration.
- `docker compose up -d --build` rebuilds images before starting or updating services; `-d` runs them in the background.

Display Logs

```
docker compose logs nginx php-app
```

This command displays logs from the NGINX and PHP-FPM services.

Shutting Down Compose

`docker compose down` stops and removes this project's containers and default network. It keeps named volumes unless you add `-v` .

```
docker compose down
```